

BIS AI ASSISTANT

Complete Software Requirements Specification (SRS)

Architecture • UML • DFD • ERD • RAG • Agentic AI • Database • APIs • Security •
Build & Deployment

Item	Details
Problem Statement	26107 — AI-Powered Intelligent Assistant for Indian Standards and BIS Services
Organization	Ministry of Consumer Affairs, Food & Public Distribution
Department	Department of Consumer Affairs (DoCA)
Category	Software
Theme	Smart Automation
Document type	A-to-Z engineering blueprint and implementation-ready SRS
Baseline	User-provided PS + uploaded team architecture/execution plan

Source basis: SIH Problem Statement 26107 supplied in the conversation + the team's uploaded Final Architecture and Execution Plan. This document does not replace official BIS documentation.

Purpose: Provide one authoritative project document that explains what the system solves, how it works, how it is built, how components interact, how data moves, how AI is grounded, and how the team can implement, test, deploy and demonstrate it.

Important scope note: Items explicitly present in the supplied architecture/execution plan are treated as the project baseline. Engineering details added to make the SRS complete are marked as proposed design where they were not specified by the source.

0. Document Control & Baseline

Field	Value
Document	BIS AI Assistant — Complete SRS & Architecture Blueprint
Version	1.0 — consolidated build blueprint
Problem Statement	SIH 26107
Primary users	Consumers, MSMEs, startups, students/researchers
Primary interaction	Natural-language conversational assistant
Primary backend	Node.js + Express API Gateway
AI service	FastAPI (Python) AI / Generative AI microservice
Vector layer	Pinecone
Application data	MongoDB Atlas
Web	React / PWA
Mobile	React Native
Source baseline	Uploaded “BIS AI Assistant — Final Architecture & Execution Plan”

The supplied team architecture describes a two-service architecture: Node.js/Express for gateway and core business logic, and FastAPI/Python for AI/GenAI; both use the data/vector layer. It also defines the rationale of separation of concerns, security, independent scaling and frontend independence.

The supplied execution plan also defines the major risks: cross-service reliability, latency, two deployments, RAG hallucination, NLP learning curve, data coverage, multilingual accuracy, environment conflicts and late integration.

Reference Architecture Supplied by the Team

The following diagram is the team's supplied baseline architecture. The rest of this document refines it into implementation-level service boundaries, data flows, UML models, security controls, APIs, testing and deployment details.

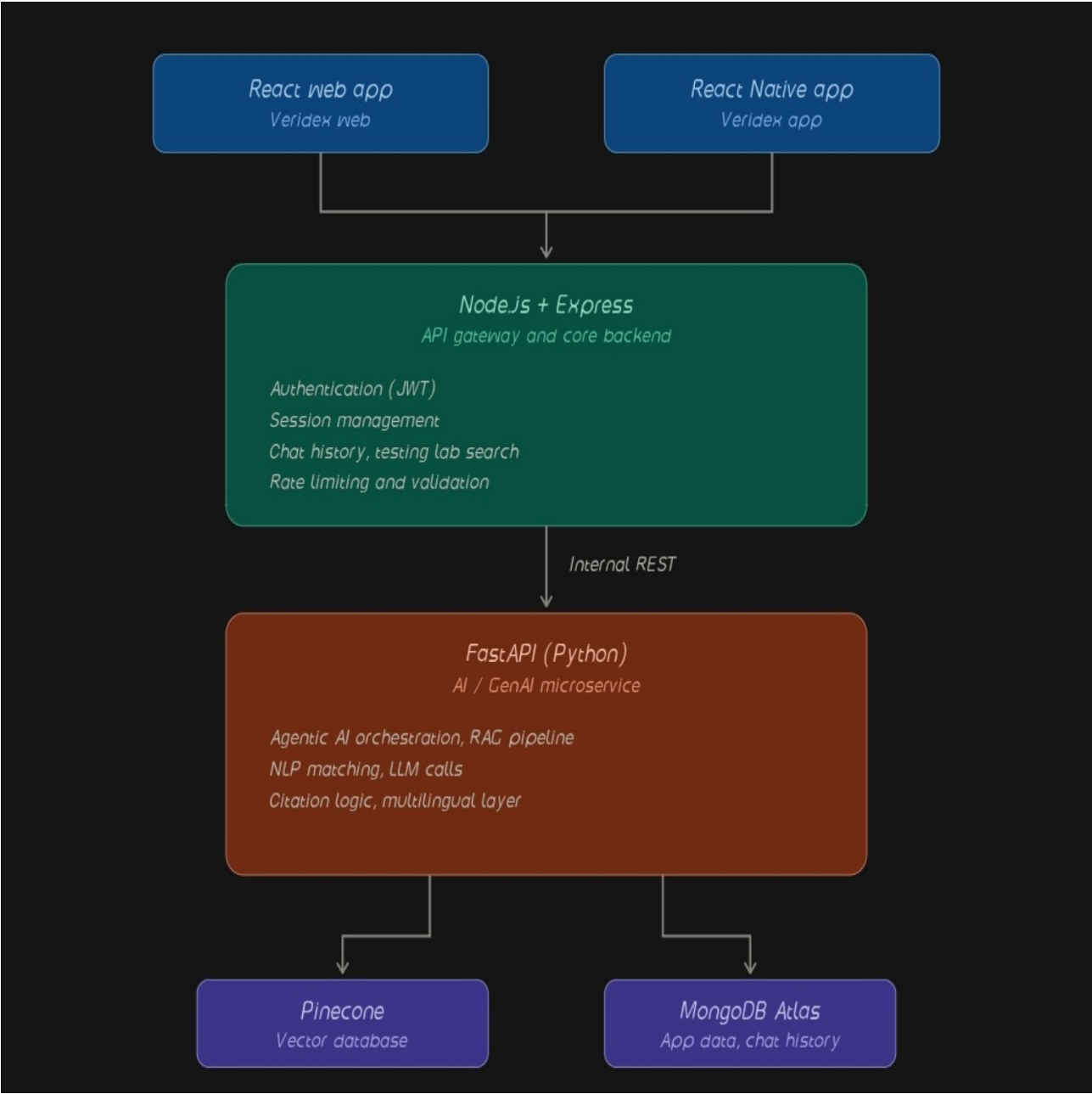


Figure 0.1 — Supplied baseline architecture. Terminology and labels have been standardized in the detailed diagrams below.

1. Contents / Document Map

Sec.	Section
2	Executive Summary & Problem Definition
3	Goals, Scope, Users & Success Criteria
4	Functional Requirements
5	Non-Functional Requirements
6	System Context & Complete Architecture
7	Application / Service Architecture
8	AI, RAG & Agentic Architecture
9	Data Architecture & Knowledge Pipeline
10	Data Flow Diagrams (Context, Level 0, Level 1, Level 2)
11	UML Use Case Diagram & Use Cases
12	UML Activity Diagram
13	UML Sequence Diagrams
14	UML Class Diagram
15	ER / Database Relationship Diagram
16	Component & Deployment Architecture
17	API & Integration Contract
18	Security Architecture
19	Multilingual Architecture
20	Hallucination Prevention & Citation Architecture
21	Testing, QA & Evaluation
22	Failure Handling, Observability & Recovery
23	Development / Build Process A→Z
24	Repository & Folder Structure
25	Team Ownership & Work Breakdown
26	Deployment & CI/CD
27	Demo Flow & Judge Questions
28	Risks, Limitations & Future Scope
29	Implementation Checklist
30	Appendix: Traceability Matrix & Glossary

2. Executive Summary & Problem Definition

The problem statement describes a landscape where the Bureau of Indian Standards publishes thousands of Indian Standards and provides services such as product certification, hallmarking, laboratory recognition, Standards Clubs, training, consumer affairs and conformity assessment. Users can struggle to identify applicable standards, certification requirements, relevant BIS schemes, licensing procedures, testing requirements, related standards and answers to technical queries.

The proposed solution is an AI-powered conversational assistant that understands plain-language queries, retrieves relevant information from authorized BIS knowledge sources and provides context-aware, source-backed answers with references to documents or clauses where applicable.

Core principle: the assistant is not designed as a generic free-form chatbot. Its high-value behavior is grounded retrieval + controlled tools + evidence-linked answers.

The supplied execution plan explicitly positions the AI layer around agentic orchestration, RAG, NLP/matching, LLM calls and citation logic, while the Node gateway provides authentication, sessions, chat history, testing laboratory search, validation and rate limiting.

What problem is being solved?

- Fragmented discovery: users have to search multiple documents, portals and PDFs.
- High cognitive load: standards and certification terminology is difficult for non-experts.
- Product-to-standard ambiguity: a user may describe a product without knowing the relevant standard number.
- Process uncertainty: certification, testing and hallmarking workflows are not always easy to navigate.
- Evidence gap: a conversational answer without a source is not sufficient for a standards/compliance use case.
- Language barrier: users may ask questions in Indian languages while the underlying source material is technical.

Proposed solution in one line

Natural-language query → intent understanding → controlled agent/tool selection → authorized BIS retrieval → grounded generation → citation validation → user-friendly answer.

3. Goals, Scope, Users & Success Criteria

3.1 Goals

- Reduce the effort required to discover relevant BIS standards and services.
- Provide source-backed conversational answers.
- Recommend applicable standards from product descriptions.
- Guide users through certification and related BIS processes.
- Help locate relevant testing laboratories using available structured data.
- Support multilingual interaction with domain-aware terminology handling.
- Maintain chat history and session continuity.
- Create a modular architecture that can scale the AI service independently.

3.2 User Personas

Persona	Needs	Typical queries
Consumer	Product safety, hallmarking, complaints/services	"How do I verify a hallmark?"
MSME / Startup	Applicable standards, certification, testing	"Which standard applies to my product?"
Student / Researcher	Standard concepts, references, related standards	"Explain this BIS standard topic."
Industry / Technical User	Compliance-oriented discovery and evidence	"What testing is associated with this category?"
Admin / Knowledge Manager	Indexing, updates, source health, content updates	"When was this source indexed?"

3.3 In Scope

- Conversational BIS knowledge search.
- Standard recommendation from product descriptions.
- Certification guidance.
- Testing-lab recommendation/search.
- Hallmarking guidance.
- Consumer-related queries.
- Multilingual input/output.
- Citations and source metadata.
- Authentication, sessions and chat history.
- Web/PWA and React Native clients.

3.4 Out of Scope / Guardrails

- The assistant should not invent a standard, clause, license requirement or laboratory.
- The assistant should not claim complete BIS coverage if the indexed dataset is partial.
- The assistant should not replace formal BIS decisions or official certification/legal processes.
- Fully autonomous open-ended agents are intentionally avoided; the source plan recommends a small fixed tool set for reliability.

3.5 Success Criteria

Metric	Target / measurement approach
Grounded answer rate	High; measure against 50+ evaluation queries and evidence support

Metric	Target / measurement approach
Citation validity	Each factual compliance/standard claim should map to retrieved evidence where applicable
Hallucination rate	Track unsupported answers separately; tune prompts/retrieval
Endpoint reliability	Health checks + timeout/fallback behavior
User task completion	Measure whether users can identify standard/service path from a query
Multilingual quality	Evaluate supported languages on technical/domain vocabulary

4. Functional Requirements

ID	Requirement	Statement
FR-01	User Query	System shall accept natural-language questions through web/mobile chat.
FR-02	Language Detection	System shall identify the input language and normalize it for processing.
FR-03	Intent Classification	System shall classify the request into supported capabilities.
FR-04	Standard Search	System shall retrieve relevant standards/knowledge from the indexed BIS dataset.
FR-05	Standard Recommendation	System shall recommend candidate standards from product descriptions using matching logic and evidence.
FR-06	Certification Guidance	System shall explain available certification-related information present in the dataset.
FR-07	Testing Lab Search	System shall filter/search testing laboratory data using available attributes such as category/location.
FR-08	Hallmarking	System shall answer hallmarking-related questions from grounded sources.
FR-09	Consumer Queries	System shall answer supported consumer-related BIS questions.
FR-10	Multilingual	System shall support configured languages and preserve technical terminology as safely as possible.
FR-11	Citation	System shall return source metadata/document/clause references where available.
FR-12	Chat History	System shall persist authenticated user sessions and chat messages.
FR-13	Authentication	System shall support JWT-based authentication.
FR-14	Validation	System shall validate request payloads and reject malformed/unsafe inputs.
FR-15	Rate Limiting	System shall limit abusive or excessive API usage.
FR-16	Fallback	System shall return a safe fallback when AI service/retrieval is unavailable.
FR-17	Health	Each service shall expose a health endpoint.
FR-18	Evaluation	The AI team shall maintain a QA sheet with 50+ sample queries and edge cases.

5. Non-Functional Requirements

ID	Area	Requirement
NFR-01	Security	AI service should not be directly exposed to clients; gateway controls access.
NFR-02	Reliability	Node↔FastAPI calls use timeouts and controlled fallback responses.
NFR-03	Performance	UI shows loading/typing state; streaming can be added to reduce perceived latency.
NFR-04	Scalability	AI service can be scaled independently from Node business logic.
NFR-05	Maintainability	Separate Node and Python projects with shared setup documentation.
NFR-06	Traceability	Answers should preserve evidence/source metadata.
NFR-07	Data Integrity	Application data and knowledge metadata must have stable IDs and validation.
NFR-08	Availability	Health endpoints and deployment monitoring shall be used.
NFR-09	Usability	User should be able to ask questions without knowing standard numbers.
NFR-10	Accessibility	Responsive UI and readable citation presentation.
NFR-11	Localization	Technical terms must be evaluated for translation accuracy.
NFR-12	Auditability	Important AI requests/results should have request IDs and logs (proposed design).
NFR-13	Privacy	Authentication/session data must be handled securely; secrets never stored in client code.
NFR-14	Portability	Web and mobile clients share the same REST contract.

Reference: supplied team architecture and execution plan, Architecture Rationale section.

6. System Context & Complete Architecture

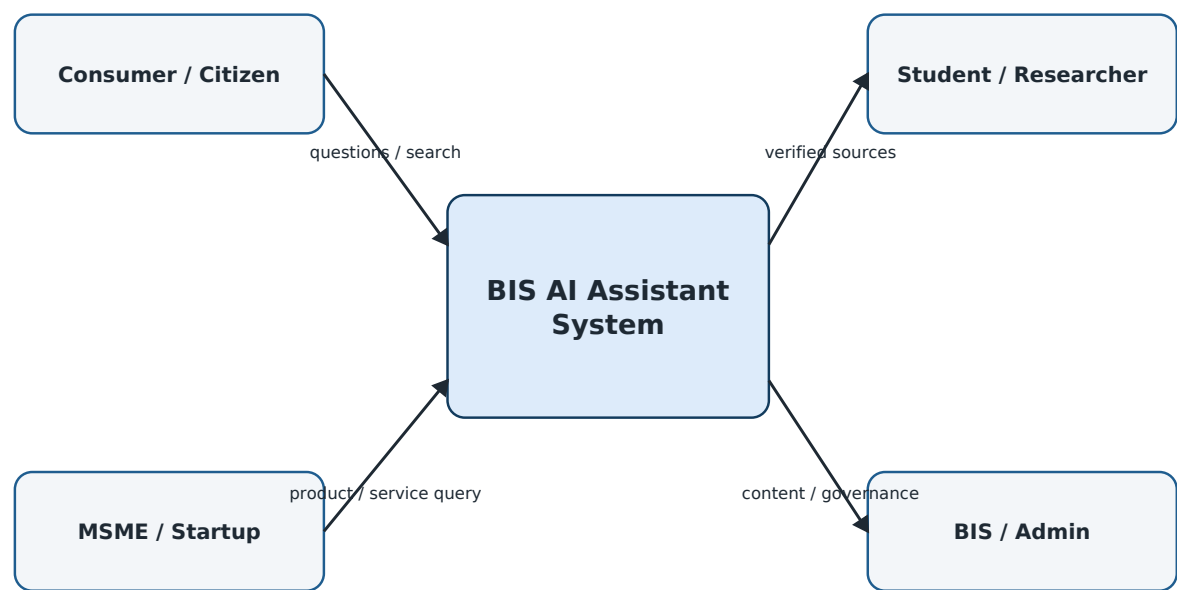


Figure 6.1 — System context: primary user groups and the BIS AI Assistant boundary.

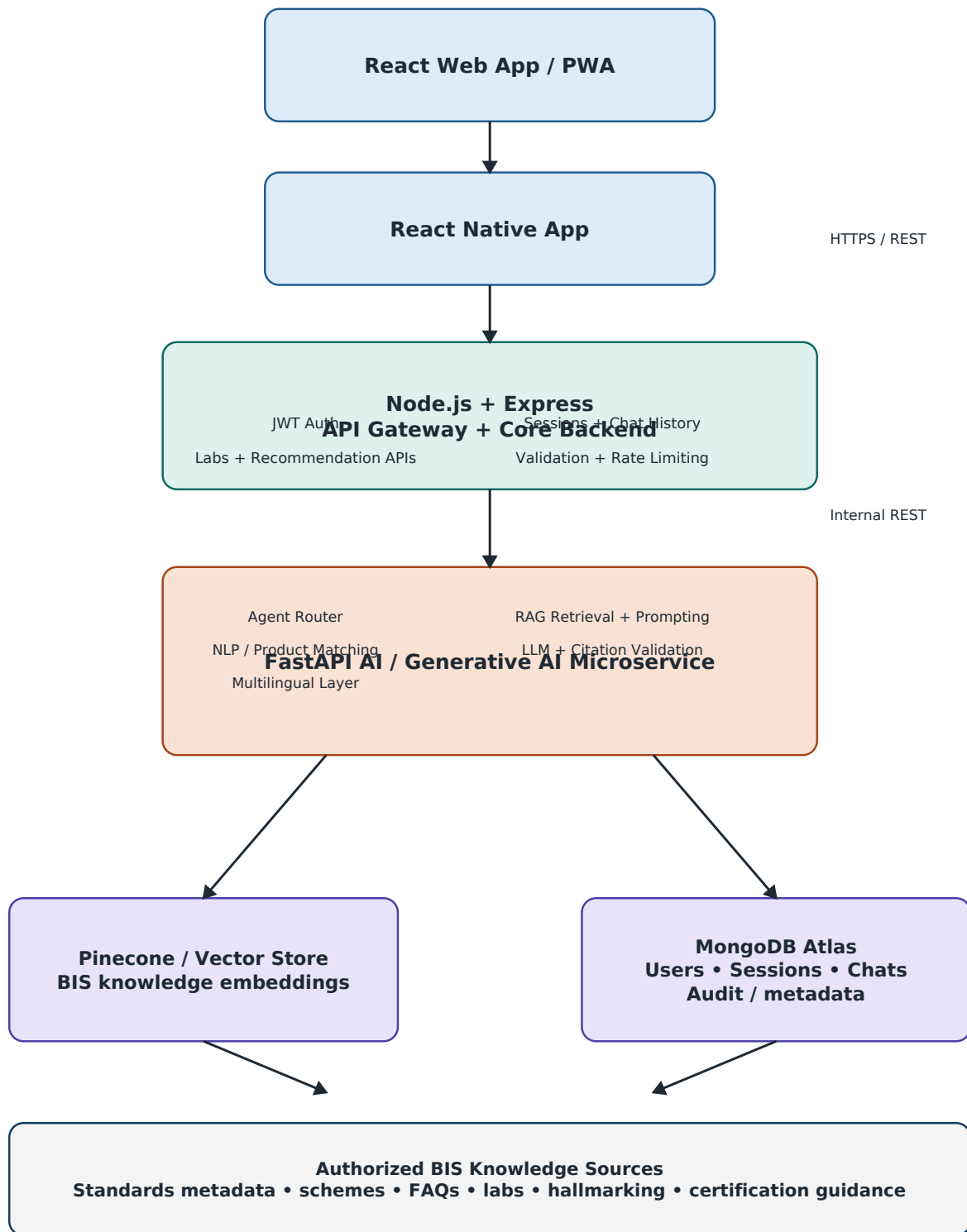


Figure 6.2 — Complete logical architecture. This expands the supplied architecture image into explicit data/knowledge boundaries.

The supplied architecture image shows React web/PWA and React Native clients feeding a Node.js + Express gateway, which calls FastAPI over internal REST. FastAPI contains agentic AI, RAG, NLP, LLM calls, citation logic and multilingual processing, while Pinecone and MongoDB Atlas provide vector and application data storage.

6.1 Architecture Principles

- Single public API contract: web and mobile use the same Node gateway.
- Private AI service: frontend never calls FastAPI directly.
- Separation of concerns: business/application logic remains in Node; AI orchestration remains in Python.
- Evidence-first generation: retrieval precedes generation for knowledge answers.
- Controlled agent: fixed tools instead of unconstrained autonomy.
- Graceful degradation: AI failures should not crash the complete backend.
- Incremental integration: connect services early with mocks before all features are finished.

The execution plan explicitly recommends early Node↔FastAPI integration with dummy responses and frontend↔Node integration with mock data to expose integration problems early.

7. Application / Service Architecture

7.1 Node.js + Express Gateway

- Public REST API entry point.
- JWT authentication and session management.
- Request validation and sanitization.
- Chat history persistence in MongoDB.
- Testing Laboratory search endpoints.
- Standard recommendation API façade.
- Rate limiting, CORS and security middleware.
- Internal FastAPI client with timeout/retry/fallback policy.
- Request ID propagation and structured logging (proposed).

7.2 FastAPI AI Service

- Agent/router orchestration.
- RAG retrieval chain.
- NLP preprocessing and entity extraction.
- Product-to-standard matching.
- Testing-lab recommendation logic.
- LLM prompt construction.
- Citation/evidence validation.
- Multilingual translation/normalization.
- AI evaluation endpoints/utilities.

7.3 Why two services?

Concern	Node.js/Express	FastAPI/Python
API gateway	Primary	Internal only
Auth/session	Primary	Not authoritative
Business CRUD	Primary	No
RAG/NLP	No	Primary
Agent orchestration	No	Primary
Vector retrieval	Via AI service/API boundary	Primary
Independent scaling	Yes	Yes

8. AI, RAG & Agentic Architecture

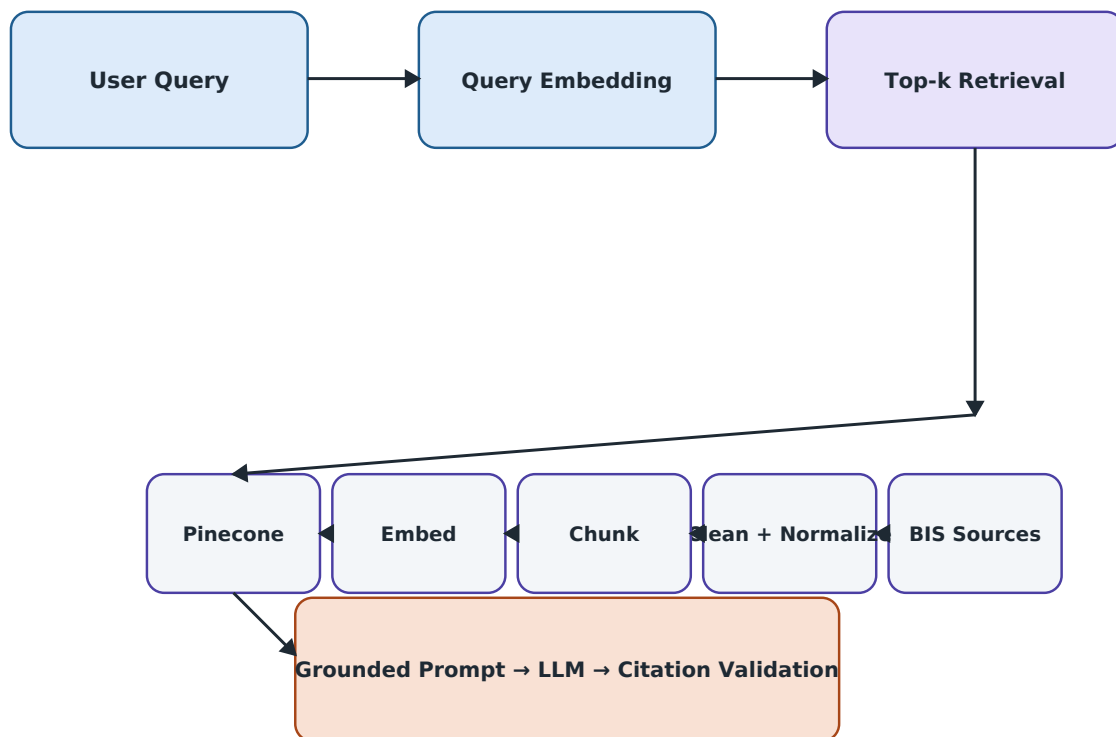


Figure 8.1 — Knowledge ingestion and RAG retrieval architecture.

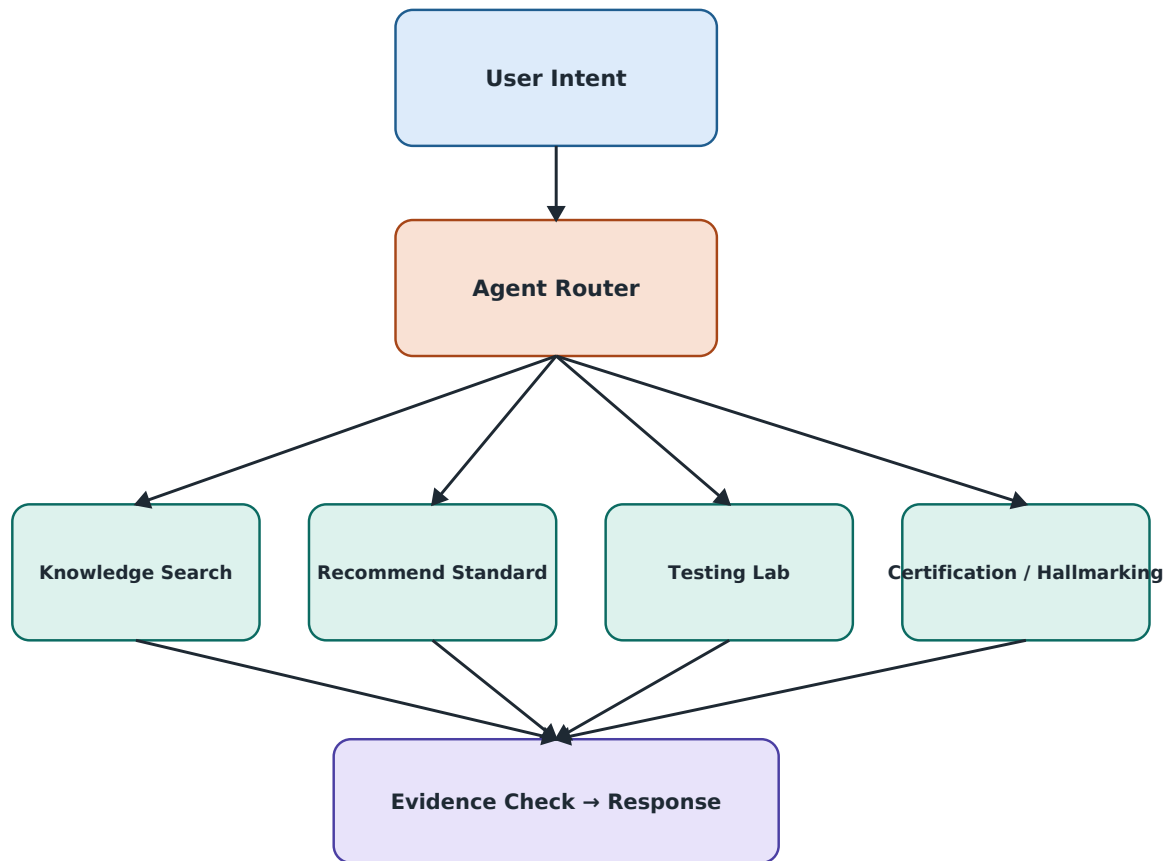


Figure 8.2 — Controlled agent router with a fixed tool set.

8.1 RAG Pipeline

- Collect authorized/allowed BIS knowledge sources.
- Clean and normalize documents.
- Attach source metadata such as title, URL, version/date and clause/page identifiers when available.
- Chunk content with overlap appropriate to the document structure.
- Generate embeddings.
- Store vectors and metadata in Pinecone.
- At query time, embed/normalize the query and retrieve top-k chunks.
- Apply relevance/evidence checks before generation.
- Generate an answer only from retrieved context.
- Attach citations to the final response.

The source execution plan explicitly calls for loading BIS data, chunking with LangChain splitters, generating embeddings, storing them in Pinecone, building similarity retrieval and mapping answers back to source documents/clauses.

8.2 Product-to-Standard Matching

- Extract product/category/attributes from the user's description.
- Normalize terms and remove irrelevant wording.

- Compute candidate matches using TF-IDF/cosine similarity as the baseline specified by the team plan.
- Optionally combine semantic retrieval with structured category filters.
- Return a ranked list, not an absolute claim, unless evidence supports the conclusion.
- Show why each candidate was suggested and cite the supporting source.

8.3 Agent Tool Contract

Tool	Input	Output
search_knowledge	query, language, filters	evidence chunks + source metadata
recommend_standard	product description, category, attributes	ranked standards + evidence
find_testing_lab	category, location, scope	matching labs
certification_guidance	product/service context	grounded guidance + sources
hallmarking_guidance	hallmark-related query	grounded guidance + sources

9. Data Architecture & Knowledge Pipeline

9.1 Logical Data Domains

Domain	Examples	Primary store
Identity	users, auth metadata	MongoDB
Conversation	sessions, messages, citations	MongoDB
Knowledge metadata	documents, versions, sources	MongoDB / dataset store
Knowledge vectors	embeddings + metadata	Pinecone
Standards catalog	standard number/title/category/status	MongoDB/dataset
Laboratories	name, location, scope/category	MongoDB/dataset
Evaluation	questions, expected evidence, observed answer	QA artifacts

9.2 Ingestion Lifecycle

- Source discovery → authorization/allowlist → acquisition → parsing → cleaning → metadata enrichment → chunking → embedding → indexing → validation → release.
- Every knowledge item should carry a stable source identity and version/date where available.
- If source content changes, re-index the affected document/version rather than silently overwriting provenance.
- The system should expose dataset coverage limitations to users when a query falls outside indexed knowledge.

The source plan identifies data quality and coverage as a major risk because actual published standards may not be fully public/free; it recommends transparently scoping the demo to genuinely covered data.

9.3 Proposed Knowledge Record

A canonical record can contain: document_id, source_name, source_url, title, standard_number, document_type, category, version, effective_date, page_or_clause, text, language, checksum, indexed_at, access_status. This is a proposed schema to make citations and re-indexing reliable.

10. Data Flow Diagrams

10.1 DFD — Context Level

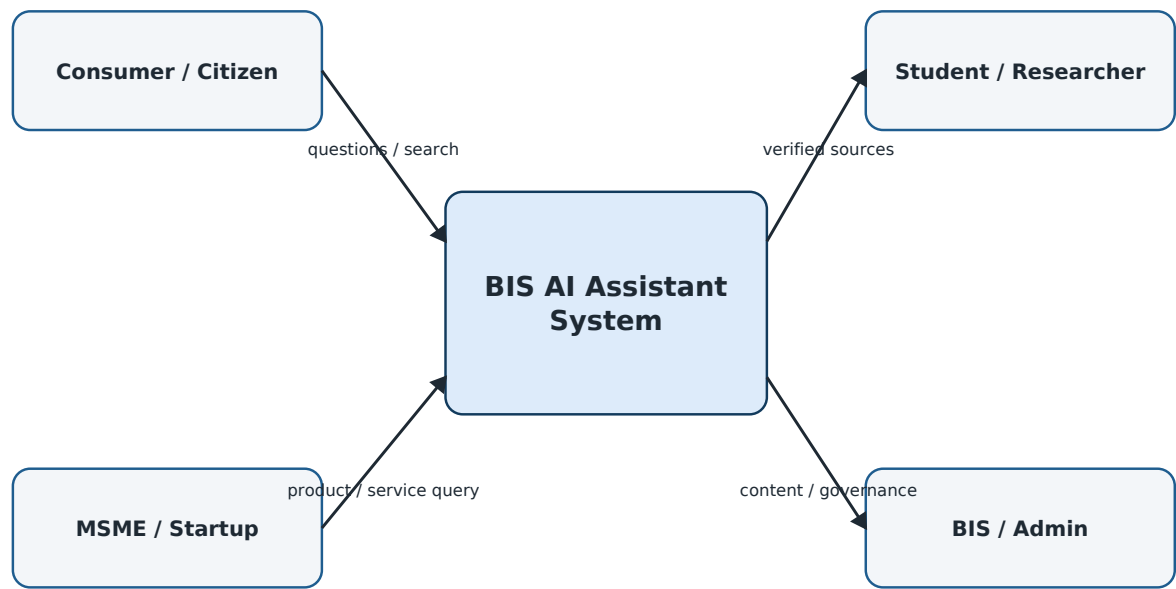


Figure 10.1 — Context DFD.

10.2 DFD — Level 0

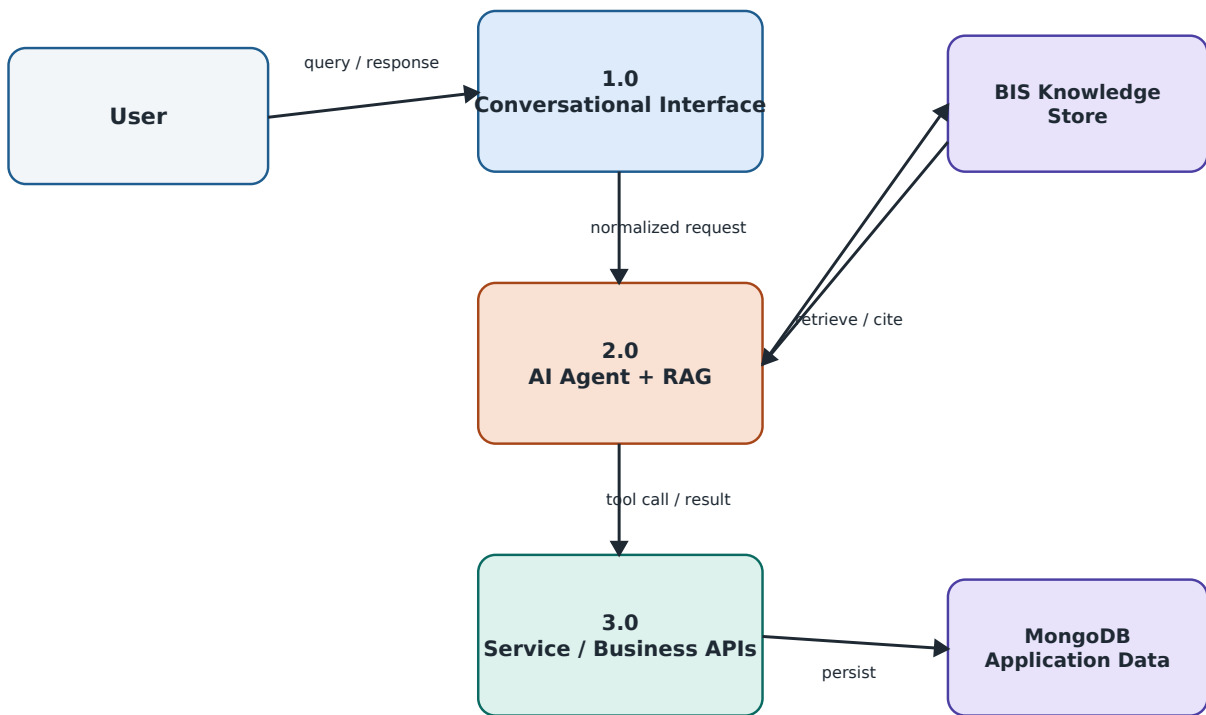


Figure 10.2 — Level 0: major processes and data stores.

10.3 DFD — Level 1: Conversational AI

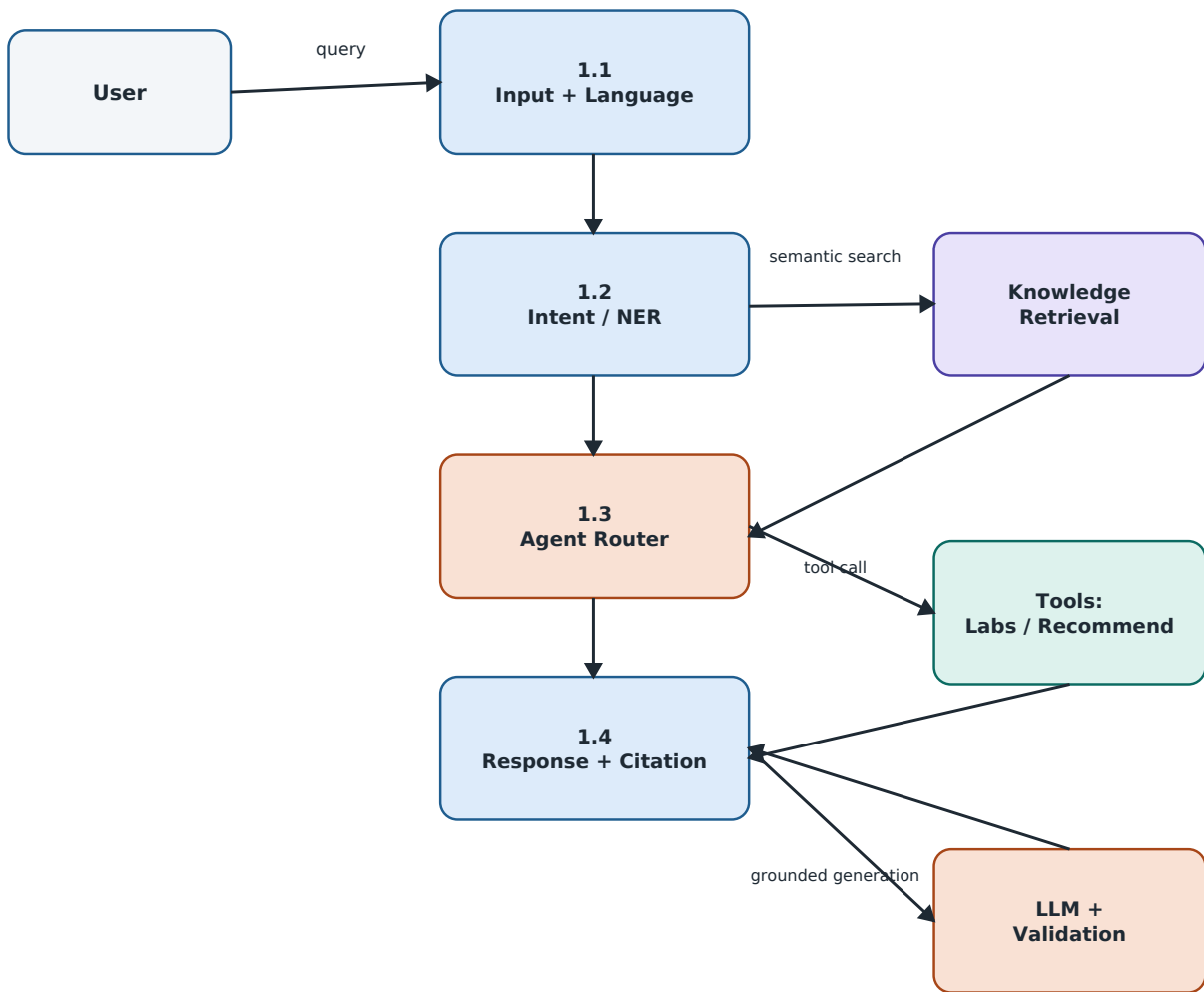


Figure 10.3 — Level 1 decomposition of query processing, routing, retrieval/tools and generation.

10.4 DFD — Level 2: RAG + Citation Flow

2.1 Query Normalization → language detection → text normalization → entity extraction

↓

2.2 Retrieval → query embedding → Pinecone top-k → metadata filtering → relevance threshold

↓

2.3 Evidence Assembly → deduplicate chunks → preserve document/section/clause metadata → build context

↓

2.4 Generation → strict grounded prompt → LLM → draft answer

↓

2.5 Citation Validation → verify claims against evidence → remove unsupported claims / fallback → final response

↓

2.6 Persistence → save message + citations + request ID → return JSON to Node → client UI

10.5 Data Flow Rules

- User input is untrusted input; validate at the gateway.
- Only normalized/authorized internal requests reach FastAPI.
- Retrieved context is treated as evidence, not as instructions.
- LLM output is not considered authoritative by itself.
- Citation metadata travels with evidence and must be preserved to the response layer.
- Chat history stores the final response and citations, not hidden model reasoning.

11. UML Use Case Diagram & Detailed Use Cases

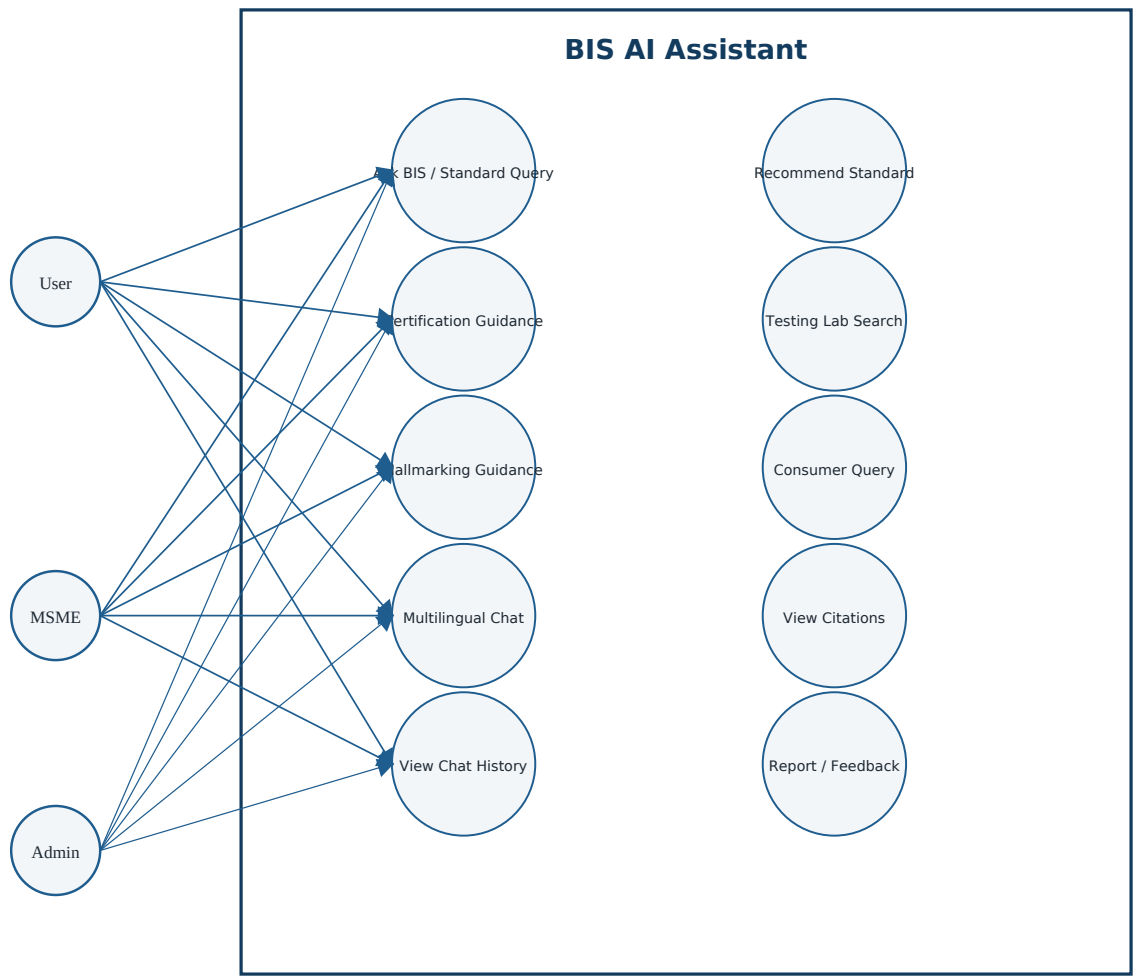


Figure 11.1 — UML-style use case model.

11.1 Core Use Case Specifications

ID	Use case	Main success path
UC-01	Ask BIS Question	User enters natural-language query; system retrieves relevant evidence and returns grounded answers.
UC-02	Recommend Standard	User describes product; system extracts features, matches candidates and returns ranked recommendations.
UC-03	Certification Guidance	User asks about certification; system retrieves relevant scheme/process information and explains requirements.
UC-04	Find Testing Lab	User provides product/category and optional location; system filters lab data and returns matches.
UC-05	Hallmarking Guidance	User asks hallmarking question; system searches the hallmarking knowledge set and cites sources.
UC-06	Multilingual Query	User asks in a supported language; system normalizes the query and responds in the selected language.
UC-07	View History	Authenticated user opens previous sessions and messages.
UC-08	View Citation	User opens a source reference attached to an answer.

12. UML Activity Diagram

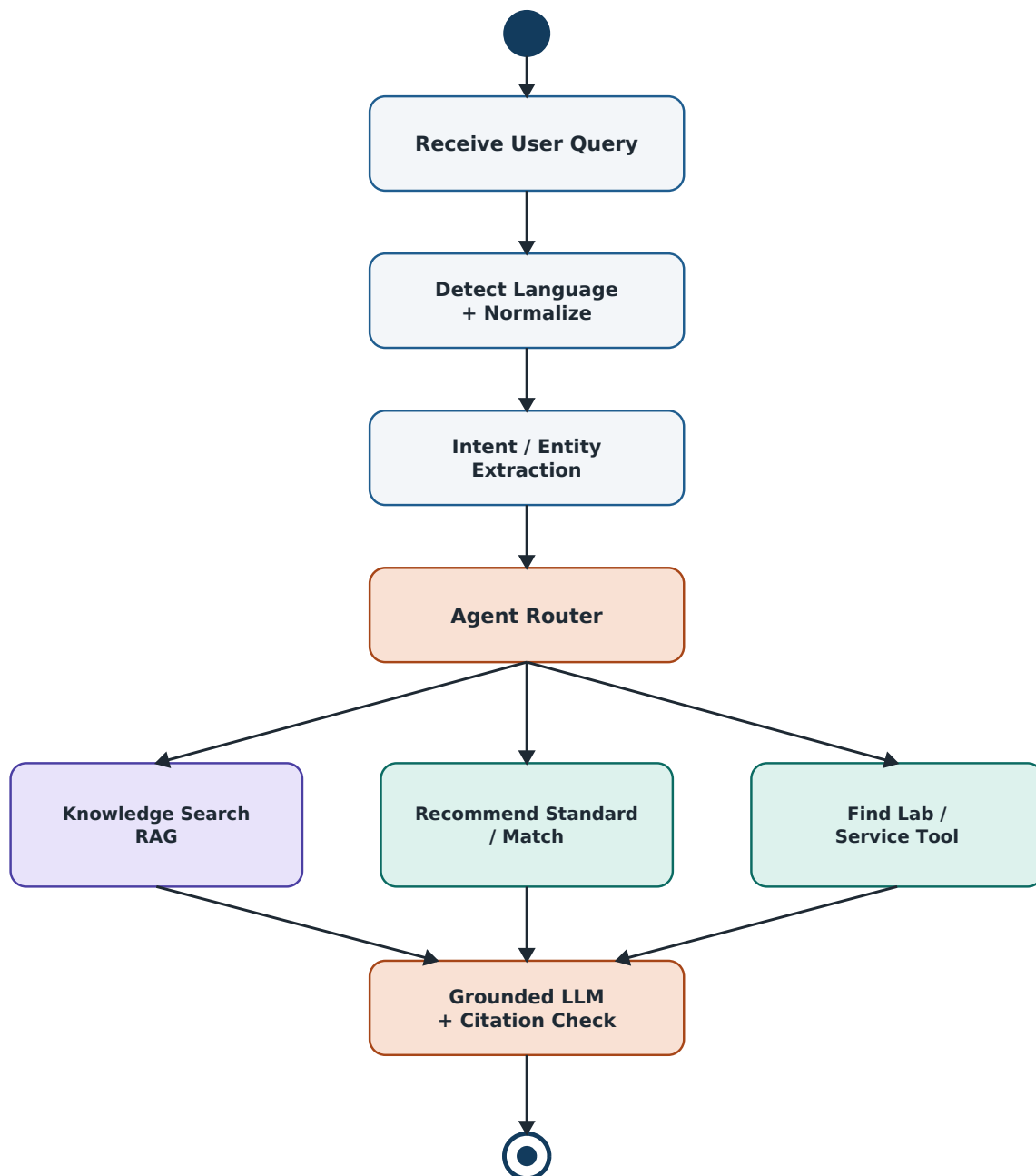


Figure 12.1 — End-to-end activity: query → route → tool/RAG → grounded response.

Activity decisions should include at least: supported/unsupported language, recognized/unrecognized intent, sufficient retrieval evidence, tool success/failure, and citation validation pass/fail.

13. UML Sequence Diagrams

13.1 Main Chat Sequence

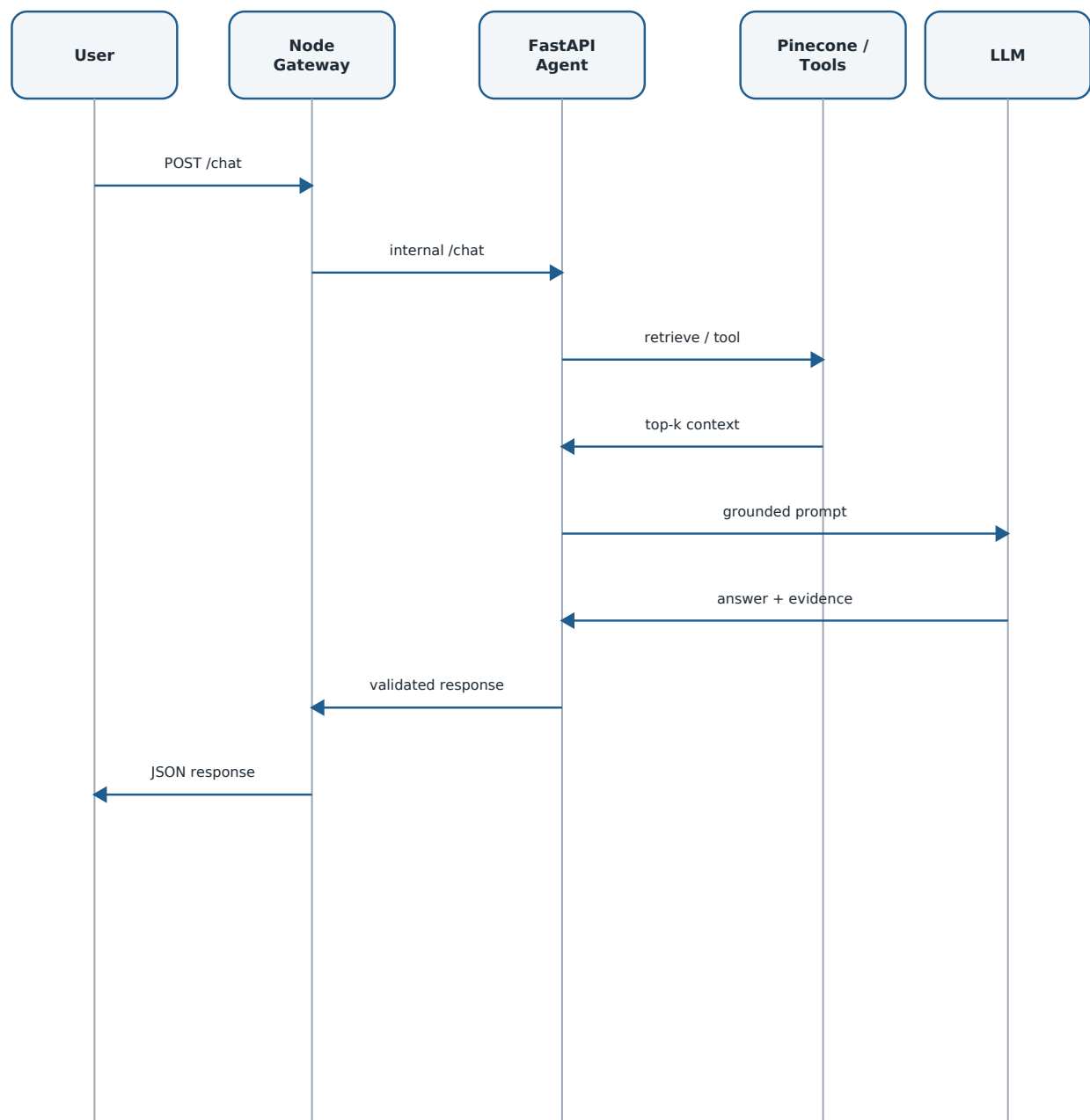


Figure 13.1 — Sequence from client to gateway, AI service, retrieval/tools and LLM.

13.2 Standard Recommendation Sequence

User → React → Node: POST /api/recommend-standard with product description.
Node → FastAPI: internal recommendation request with request ID and normalized data.
FastAPI: extract product attributes → baseline TF-IDF/cosine matching + knowledge retrieval → rank candidates → evidence check.
FastAPI → Node: ranked standards + explanation + citations.
Node → MongoDB: persist request/result in session history.
Node → React: structured response for recommendation cards.

13.3 Failure Sequence

Client → Node → FastAPI → timeout/error → Node logs failure + returns safe message.
If retrieval returns no relevant evidence: FastAPI returns insufficient evidence state rather than fabricating an answer.
If LLM output fails citation validation: discard/repair the answer and return a grounded fallback.

14. UML Class Diagram

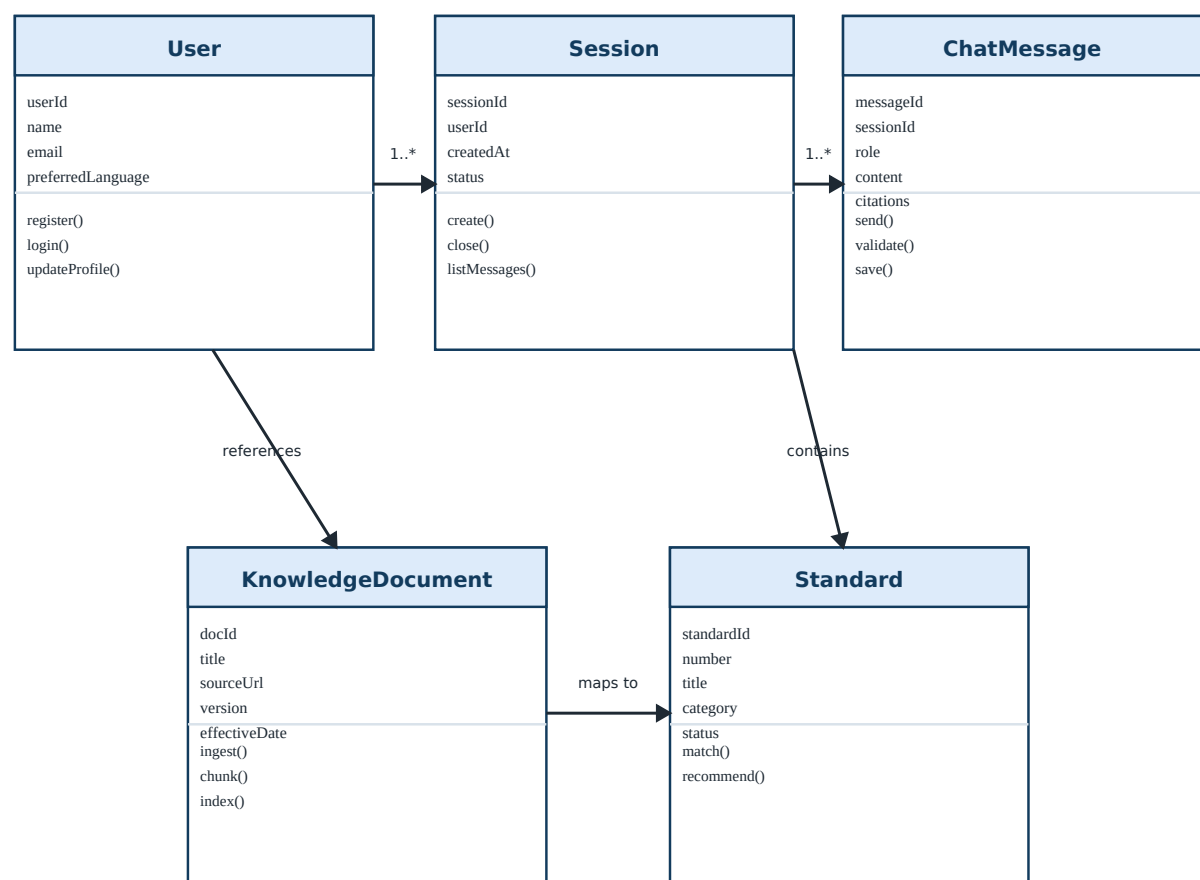


Figure 14.1 — Core domain classes and relationships.

14.1 Core Class Responsibilities

Class	Responsibility
User	Identity, profile, preferred language and authentication reference.
Session	Conversation container and lifecycle.
ChatMessage	User/assistant message, citations, timestamps and validation status.
KnowledgeDocument	Source document metadata and ingestion lifecycle.
Standard	Structured standard catalog entity and recommendation target.
TestingLab	Structured laboratory entity used by the testing laboratory tool.
Citation (proposed)	Evidence pointer containing document/chunk/page/clause metadata.
AgentRequest (proposed)	Internal AI orchestration request with intent/tool selection and request ID.

15. ER / Database Relationship Diagram

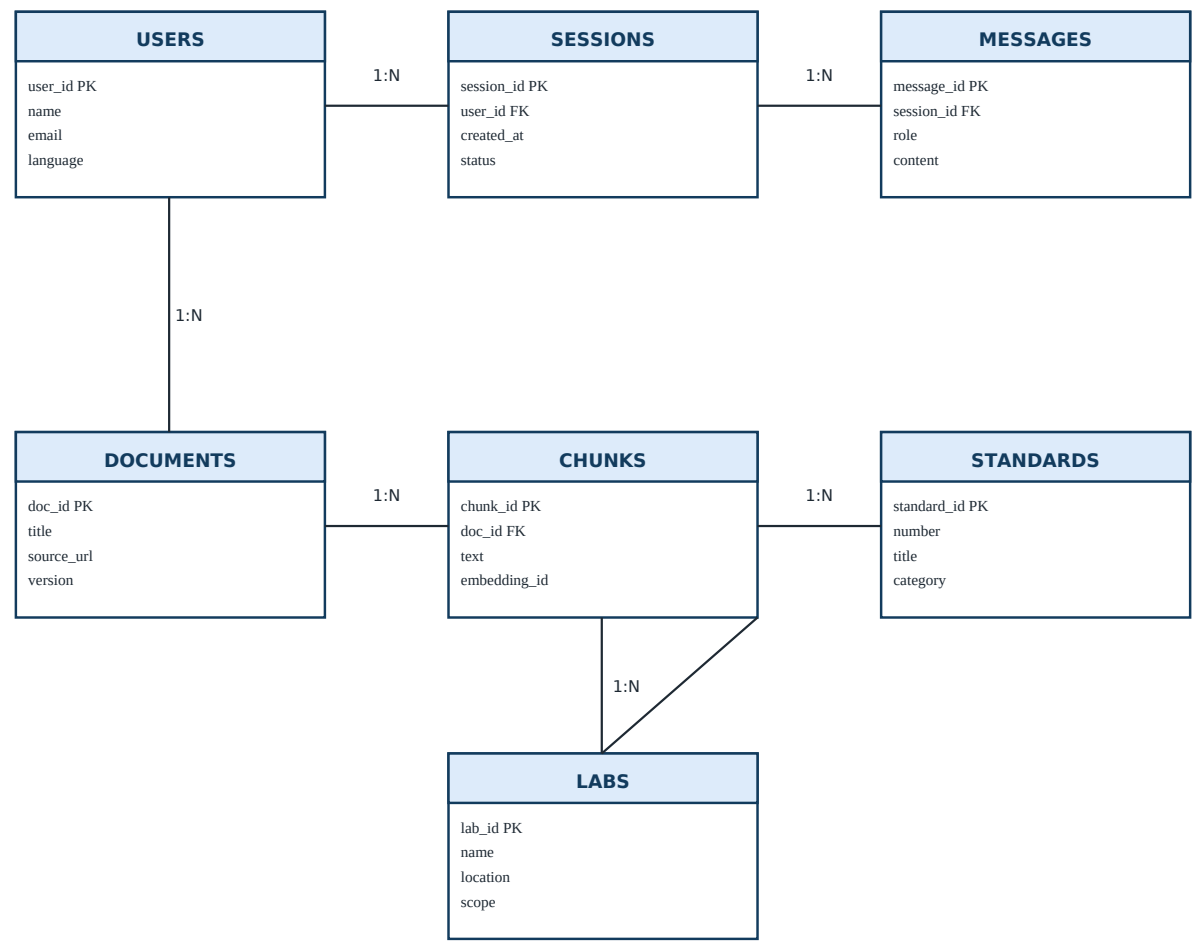


Figure 15.1 — Proposed relational-style entity relationship view for the MongoDB domain model.

MongoDB is document-oriented, so the ER diagram is a logical relationship model rather than a requirement to physically normalize collections exactly as a SQL schema.

15.1 Proposed Collections

Collection	Key fields	Purpose
users	_id, email, name, language, createdAt	Identity/profile
sessions	_id, userId, title, status, createdAt	Conversation sessions
messages	_id, sessionId, role, content, citations, createdAt	Chat history
documents	_id, sourceUrl, title, version, checksum, indexedAt	Knowledge metadata
standards	_id, number, title, category, status	Structured standards catalog
labs	_id, name, location, scope, categories	Testing-laboratory search
feedback	_id, userId, messageId, rating, comment	Quality feedback (proposed)
audit_events	_id, requestId, type, timestamp, metadata	Operational trace (proposed)

16. Component & Deployment Architecture

16.1 Component Architecture

Web/Mobile UI contains Chat UI, Standard Recommendation UI, Testing Laboratory Finder, Language Selector, History and Citation components.

↓

API Gateway contains Auth Controller, Chat Controller, Recommendation Controller, Lab Controller, Session Controller and AI Client.

↓

AI Service contains Agent Router, RAG Service, Retrieval Client, Matching Service, Lab Tool, Prompt Builder, LLM Client, Citation Validator and Translation Layer.

↓

Data Services contain MongoDB repositories and Pinecone retrieval/indexing clients.

16.2 Deployment Architecture

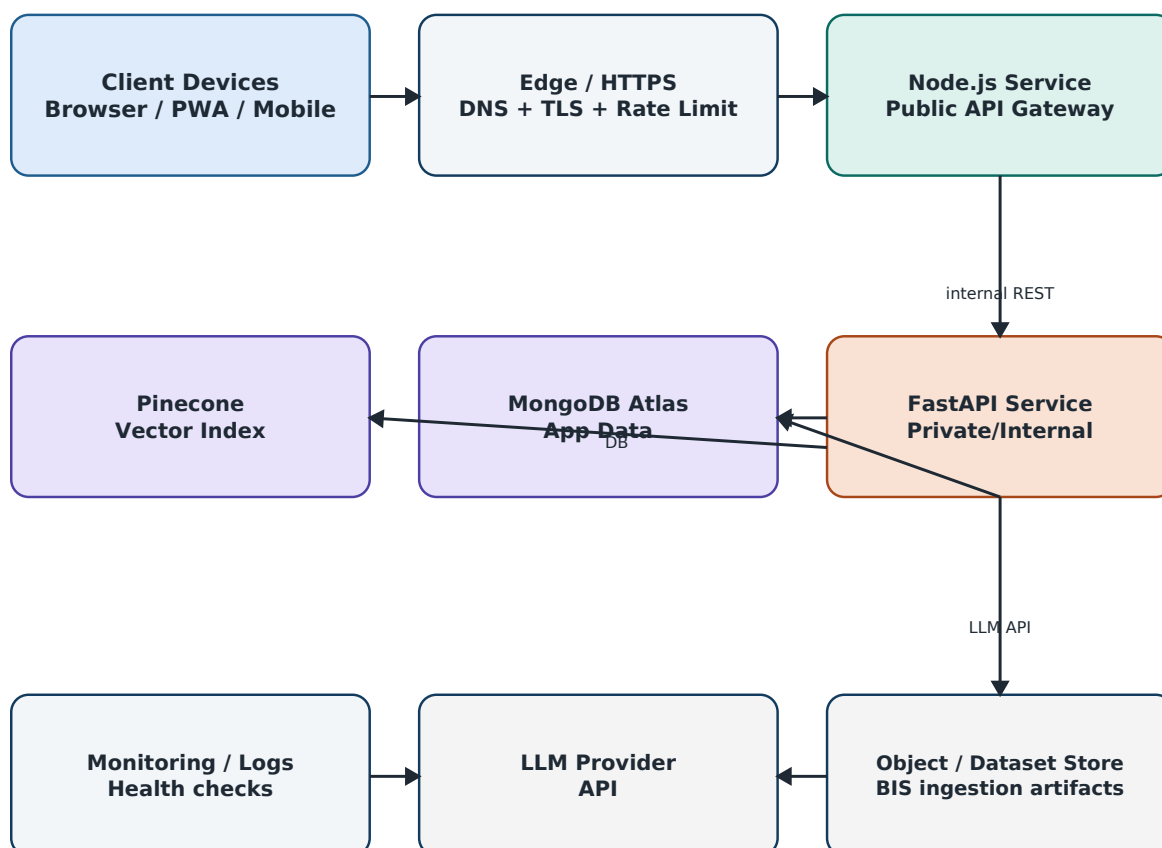


Figure 16.1 — Proposed deployment topology.

The supplied plan recommends separate deployments for Node and FastAPI, health endpoints for both, and keeping the AI service internal/private behind the gateway.

17. API & Integration Contract

17.1 Public API (Node Gateway)

Method	Endpoint	Purpose
POST	/api/auth/register	Create user
POST	/api/auth/login	Issue JWT/session
POST	/api/sessions	Create chat session
GET	/api/sessions	List user sessions
GET	/api/sessions/:id/messages	Get history
POST	/api/chat	Send conversational query
POST	/api/recommend-standard	Product-to-standard recommendation
GET	/api/labs	Testing-laboratory search/filter
POST	/api/feedback	Submit response feedback
GET	/health	Service health

17.2 Internal FastAPI API

Method	Endpoint	Purpose
POST	/api/chat	AI orchestration + grounded response
POST	/api/recommend	Standard recommendation
GET/POST	/api/labs	Lab recommendation/search support
GET	/health	AI service health

The supplied execution plan names /chat, /sessions, /labs and /recommend-standard on the Node side and /api/chat, /api/recommend and /api/labs on FastAPI, plus /health.

17.3 Example Chat Request

```
POST /api/chat
{ "sessionId": "...", "message": "Which BIS standard may apply to my product?", "language": "en" }
Response (proposed):
{ "answer": "...", "citations": [{"documentId": "...", "title": "...", "pageOrClause": "..."}], "intent":
"standard_query", "requestId": "..." }
```

17.4 Internal Service Contract Rules

- Internal requests must carry a request/correlation ID.
- FastAPI must not trust client-provided authorization context beyond what Node forwards.
- Timeouts and explicit error codes are required.
- Response schema must distinguish grounded success, insufficient evidence and service failure.
- No raw secrets or provider keys are sent from clients.

18. Security Architecture

18.1 Security Controls

- HTTPS/TLS for all client traffic.
- JWT authentication at the gateway.
- Secure password hashing if passwords are stored (proposed implementation detail).
- CORS allowlist and security headers such as Helmet.
- Rate limiting for public endpoints.
- Payload validation and maximum input length.
- Prompt-injection defense: retrieved documents are evidence, not executable instructions.
- Secrets in environment/secret manager, never in repository.
- AI service network access restricted to gateway/internal callers.
- PII minimization in logs; never log passwords/tokens.
- Audit request IDs for incident tracing.
- Dependency scanning and pinned/controlled versions.

18.2 Threat Model

Threat	Risk	Mitigation
Prompt injection	LLM follows malicious content	Treat retrieved text as data; strict system prompt; tool allowlist
Hallucination	Incorrect compliance advice	RAG + evidence threshold + citation validation
API abuse	Excessive requests	Rate limiting + auth + validation
Token theft	Account compromise	HTTPS + secure token storage + expiry/rotation strategy
Data poisoning	Bad knowledge indexed	Source allowlist + ingestion validation + versioning
Service outage	Chat unavailable	Timeouts + fallback + health checks
Sensitive logs	Privacy leakage	Structured redacted logs

19. Multilingual Architecture

Input language → language detection → technical term preservation → optional translation/normalization → intent/retrieval → grounded answer → response translation → citation rendering.

The source plan warns that machine translation can introduce domain-specific errors and recommends testing technical vocabulary and limiting the demo to languages that test well rather than making unverified broad claims.

19.1 Design Rules

- Keep standard numbers, scheme names and clause identifiers unchanged unless a verified localized label exists.
- Use a controlled glossary for key BIS terms.
- Evaluate multilingual answers against English/source evidence.
- If translation confidence is low, show the source language or an explicit limitation.
- Store the user's preferred language in the session/profile.

20. Hallucination Prevention & Citation Architecture

20.1 Evidence-First Answer Policy

- Retrieve evidence before generation for factual BIS knowledge queries.
- Use a relevance threshold; if insufficient, ask a clarifying question or say information is unavailable.
- Prompt the LLM to answer only from supplied evidence.
- Require source metadata in the generation context.
- Post-check generated claims against evidence/citation mapping.
- Return an uncertainty/coverage message when the indexed dataset cannot support the answer.

20.2 Citation Object (proposed)

```
{ "documentId": "...", "title": "...", "sourceUrl": "...", "page": 12, "clause": "4.2", "snippet": "...", "version": "...", "retrievedAt": "..." }
```

20.3 Citation UI

- Inline source markers beside relevant claims.
- Expandable source card with document title and clause/page.
- Source URL/open action when a legitimate accessible URL exists.
- Show source version/date when available.
- Never present an unverified citation as official.

21. Testing, QA & Evaluation

21.1 Testing Pyramid

Level	Examples
Unit	NLP normalization, similarity scoring, citation validator, API validators
Integration	Node→FastAPI, FastAPI→Pinecone, FastAPI→LLM, Node→MongoDB
End-to-End	User query through UI to cited response
Security	Auth, rate limit, malformed payloads, prompt injection
Performance	Concurrent chat requests, retrieval latency, endpoint load
Multilingual	Hindi/English and other explicitly supported languages
Regression	50+ curated questions + edge cases

The source plan requires a RAG evaluation/QA sheet with 50+ sample queries and testing of empty, long, off-topic and non-English queries.

21.2 Evaluation Dataset Template

Case	Expected	Evidence required	Observed	Pass?
Standard lookup	Relevant standard facts	Yes	—	—
Product recommendation	Ranked candidates	Yes	—	—
Certification	Correct process guidance	Yes	—	—
Hallmarking	Grounded explanation	Yes	—	—
Testing Laboratory	Relevant laboratory/laboratories	Structured data	—	—
Unsupported topic	Safe refusal/coverage notice	No unsupported claim	—	—
Adversarial prompt	No policy bypass	N/A	—	—

22. Failure Handling, Observability & Recovery

22.1 Failure Matrix

Failure	Detection	System behavior
FastAPI down	HTTP timeout/connection error	Node returns safe AI-unavailable response; log request ID
LLM timeout	Provider timeout	Retry only where safe; fallback/insufficient evidence response
Pinecone unavailable	Vector query exception	Do not fabricate; return temporary knowledge-unavailable state
No evidence	Low retrieval score/empty result	Ask clarification or say no supported information
Citation mismatch	Validator failure	Regenerate with stricter prompt or suppress unsupported claim
MongoDB unavailable	DB exception	Do not lose user-facing request; return controlled error and retry/persist if safe
Rate limit	Threshold exceeded	HTTP 429 with retry guidance
Bad input	Schema validation	Reject before AI service

22.2 Health Endpoints

Node /health: API process, database connectivity and dependency status (as appropriate). FastAPI /health: service process, vector/LLM dependency readiness. A distinction between liveness and readiness is recommended for production.

22.3 Observability

- Correlation/request ID across client → Node → FastAPI.
- Structured logs with latency, status, intent/tool and error code.
- Metrics: request count, latency, retrieval hit rate, evidence threshold failures, citation validation failures, LLM errors.
- Dashboard/alerts are proposed for deployment; the supplied source only mandates health checks and shared deployment documentation.

23. Development / Build Process A→Z

23.1 Phase 0 — Freeze Requirements

- Freeze supported features and demo languages.
- Define authoritative/authorized data sources.
- Define citation metadata format.
- Define 50+ evaluation queries before model tuning.

23.2 Phase 1 — Repository & Environments

- Create separate frontend, mobile, node-backend and ai-service projects or a monorepo with clear boundaries.
- Pin Node/Python versions.
- Create .env.example files.
- Add README with local setup and service URLs.
- Create health endpoints immediately.

23.3 Phase 2 — Data First

- Collect a small, clean, verified subset first.
- Create parser/cleaner.
- Create chunker and metadata schema.
- Generate embeddings and index Pinecone.
- Run retrieval-only tests before adding the LLM.

23.4 Phase 3 — FastAPI AI Core

- Implement query normalization.
- Implement retrieval chain.
- Implement strict grounded prompt.
- Implement citation object.
- Implement agent router with fixed tools.
- Implement recommendation baseline and laboratory search.
- Add multilingual layer after English retrieval works.

23.5 Phase 4 — Node Gateway

- Implement auth/session APIs.
- Implement MongoDB models/repositories.
- Implement /api/chat and internal FastAPI client.
- Add /recommend-standard and /labs.
- Add validation, security middleware, rate limiting.
- Add timeout/fallback.

23.6 Phase 5 — Frontend + Mobile

- Build chat UI.
- Build citation component.

- Build recommendation form/cards.
- Build laboratory search.
- Build language selector.
- Build history.
- Integrate live APIs only after mock contracts are stable.

23.7 Phase 6 — Integration + QA

- Run complete flows daily.
- Execute 50+ evaluation cases.
- Fix retrieval before prompt cosmetics.
- Measure latency and failure paths.
- Run mobile + web demo rehearsal.

23.8 Phase 7 — Deployment

- Deploy Node gateway.
- Deploy FastAPI separately/private.
- Deploy React/PWA.
- Build React Native APK with Expo EAS.
- Configure production environment variables.
- Verify /health endpoints.
- Record backup demo.

The supplied critical path is Data Collection → FastAPI/RAG/Agentic AI → Node Gateway Integration → Frontend/App Integration → QA → Deployment → Presentation.

24. Repository & Folder Structure

24.1 Recommended Monorepo Layout

```
bis-ai-assistant/  
├── apps/  
│   ├── web/ # React / PWA  
│   └── mobile/ # React Native / Expo  
├── services/  
│   ├── gateway/ # Node.js + Express  
│   └── ai-service/ # FastAPI + RAG + Agent  
├── data/  
│   ├── raw/ # controlled local artifacts  
│   ├── cleaned/  
│   ├── schemas/  
│   └── evaluation/  
├── docs/  
│   ├── architecture/  
│   ├── api/  
│   └── diagrams/  
├── scripts/  
│   ├── ingestion/  
│   └── evaluation/  
├── .env.example  
├── README.md  
└── docker-compose.yml # optional local orchestration
```

24.2 FastAPI Internal Modules

```
ai-service/  
├── app/  
│   ├── main.py  
│   ├── api/  
│   ├── agent/  
│   ├── rag/  
│   ├── retrieval/  
│   ├── matching/  
│   ├── citations/  
│   ├── multilingual/  
│   ├── tools/  
│   ├── prompts/  
│   └── schemas/  
├── ingestion/  
├── evaluation/  
└── tests/
```

24.3 Node Internal Modules

```
gateway/  
├── src/  
│   ├── routes/  
│   ├── controllers/  
│   ├── services/  
│   ├── models/  
│   ├── middleware/  
│   ├── validators/  
│   ├── clients/aiClient.js  
│   ├── config/  
│   └── utils/  
└── tests/
```

25. Team Ownership & Work Breakdown

Area	Owner(s)	Key deliverables
Node backend / gateway	Deep, Anurag	Express APIs, auth, sessions, AI client, validation, security
MongoDB	Deep, Anurag	Schemas, repositories, chat history
FastAPI	Atharv, Pranay	Service setup, endpoints, health
RAG	Atharv, Pranay	Ingestion, chunking, embeddings, retrieval
Agentic AI	Atharv, Pranay	Fixed tool router and orchestration
NLP / matching	Atharv, Pranay	TF-IDF/cosine baseline, preprocessing
Pinecone	Atharv, Pranay	Vector index and retrieval
React frontend	Khushi, Niharika	Chat, citations, forms, responsive UI
React Native	Deep	Chat, history, language selector, APK
Data collection	Khushi, Niharika	BIS/eBIS source collection and cleaning
QA / evaluation	Atharv, Pranay	50+ query evaluation and edge cases
Deployment	Deep lead, Anurag support	Node, FastAPI, web/app deployment
Presentation/demo	Khushi, Niharika, Pranay	PPT, queries, judge Q&A, backup video

This ownership table follows the supplied team plan.

25.1 Critical Dependency Rule

Data collection must begin immediately. The AI pipeline should start on a small verified subset rather than waiting for 100% data. This is explicitly identified as the biggest dependency risk in the supplied plan.

26. Deployment & CI/CD

26.1 Environments

Environment	Purpose
Local	Individual development; mocked external services where needed
Staging / Demo	Integrated system with representative dataset
Production (future)	Controlled BIS-scale deployment with governance

26.2 CI Pipeline

Git push → lint → unit tests → build → security/dependency checks → integration tests → artifact creation → deploy to staging → smoke tests → manual approval → production (future).

26.3 Deployment Checklist

- Set environment variables for MongoDB, Pinecone, LLM provider and JWT secret.
- Restrict CORS origins.
- Verify internal FastAPI URL is not publicly exposed to clients.
- Run /health checks.
- Run smoke query: standard search.
- Run smoke query: recommendation.
- Run smoke query: hallmarking.
- Run multilingual smoke query.
- Verify citation rendering.
- Verify chat history.
- Record backup demo video.

The supplied plan mentions Render/Railway/Vercel as deployment options and recommends shared documentation of both service URLs, environment variables and health endpoints.

27. Demo Flow & Judge Questions

27.1 Recommended Demo Story

- Open landing page and explain the fragmented-search problem.
- Ask a normal BIS/standard question and show the cited answer.
- Enter a product description and demonstrate standard recommendation.
- Ask a certification/process question.
- Use testing laboratory search with a location/category.
- Ask a hallmarking question.
- Switch language and demonstrate multilingual interaction.
- Open citation details.
- Open chat history.
- Briefly show architecture and evaluation metrics.

The supplied presentation plan explicitly recommends 3–5 strong demo queries spanning standard search, product recommendation, certification guidance, hallmarking and multilingual query, plus a backup demo video.

27.2 Judge Questions — Prepared Answers

Question	Answer
How do you prevent hallucination?	RAG retrieves evidence from the indexed BIS knowledge base; the prompt restricts generation to retrieved
Why microservices?	Node handles auth/business/API concerns while FastAPI handles AI/RAG/NLP. This allows independent scaling
How do you know an answer is correct?	The evaluation set measures grounded answers and citation/evidence support. The system is designed to a
What if BIS data is incomplete?	The system explicitly scopes its answer to indexed/authorized coverage and should say when it lacks suffici
Why not train your own LLM?	The problem is knowledge retrieval and controlled assistance, not building a foundation model. RAG provid
How does product recommendation work?	A baseline TF-IDF/cosine matching pipeline ranks product-to-standard candidates; evidence retrieval supp
Can it scale?	The AI service can scale independently, vector retrieval scales separately, and the gateway remains the st
What if AI service is down?	Node uses timeouts and controlled fallback messages; health endpoints make the failure visible.

28. Risks, Limitations & Future Scope

28.1 Known Risks

Risk	Current mitigation
Cross-service reliability	Timeouts, fallback, health endpoints
Latency	Loading state; optional streaming
RAG hallucination	Strict grounding + citation validation
Data coverage	Transparent scope and source metadata
NLP learning curve	Simple baseline methods; fixed agent tools
Multilingual errors	Domain-specific testing and limited verified languages
Integration crunch	Early mock integration
Environment conflicts	Pinned versions + .env.example + README

These risks and mitigations are aligned with the supplied execution plan.

28.2 Limitations for Hackathon Demo

- Knowledge coverage depends on the dataset that the team can legally and reliably collect/index.
- Not every standard/clauses may be available for automated retrieval.
- Recommendation should be presented as assistance, not a formal certification determination.
- Multilingual support should be limited to languages actually evaluated.
- Testing-lab results depend on the completeness and freshness of the lab dataset.

28.3 Future Scope

- Formal BIS data partnership and complete governed knowledge access.
- Document version tracking and automatic re-indexing.
- Admin knowledge management portal.
- Advanced semantic product-to-standard matching.
- More robust multilingual/domain translation.
- Feedback-driven retrieval evaluation.
- Enterprise SSO and role-based access.
- Production-grade observability and autoscaling.
- Structured compliance workflow integrations where officially supported.

29. Implementation Checklist

29.1 Architecture

Done	Checklist
☐	Freeze service boundaries: React/PWA + React Native → Node gateway → FastAPI AI → Pinecone/MongoDB.
☐	Create API contract before UI integration.
☐	Keep FastAPI private/internal.
☐	Add /health to both services.
☐	Define request/correlation ID.

29.2 Data & AI

Done	Checklist
☐	Identify authorized BIS/eBIS sources.
☐	Create cleaned JSON/CSV/text dataset.
☐	Define document metadata and citation schema.
☐	Chunk + embed + index.
☐	Validate retrieval before LLM.
☐	Implement strict grounded prompt.
☐	Implement agent router and fixed tools.
☐	Implement TF-IDF/cosine recommendation baseline.
☐	Implement citation validation.
☐	Create 50+ QA cases.
☐	Test unsupported/off-topic/long/empty/non-English queries.

29.3 Backend / Frontend / Mobile

Done	Checklist
☐	JWT auth + sessions.
☐	MongoDB chat persistence.
☐	/api/chat.
☐	/api/recommend-standard.
☐	/api/labs.
☐	React chat UI.
☐	Citation UI.
☐	Recommendation UI.
☐	Language selector.
☐	History screen.
☐	React Native chat + history.
☐	APK build.

29.4 Demo / Deployment

Done	Checklist
☐	Deploy Node.
☐	Deploy FastAPI.
☐	Deploy web/PWA.
☐	Build mobile APK.
☐	Set production environment variables.
☐	Run smoke tests.
☐	Run QA sheet.
☐	Capture screenshots.
☐	Prepare 3-5 demo queries.
☐	Record backup video.
☐	Rehearse judge questions.

30. Appendix — Traceability Matrix & Glossary

30.1 Requirement-to-Architecture Traceability

PS Need	System capability	Main components
Answer Indian Standards questions	Conversational RAG	React → Node → FastAPI → Pinecone → LLM
Recommend applicable standards	Product matching	React → Node → FastAPI matching + knowledge retrieval
Certification guidance	Certification tool/knowledge retrieval	FastAPI Agent + RAG
Consumer queries	Knowledge search	FastAPI RAG
Hallmarking	Hallmarking knowledge/tool	FastAPI Agent + RAG
Testing Laboratoryoratories	Lab search/recommender	Node API + FastAPI tool + MongoDB
Multilingual interaction	Translation/normalization layer	FastAPI multilingual layer + UI selector
Source-backed information	Citation logic	FastAPI citation validator + source metadata

30.2 Glossary

Term	Meaning
BIS	Bureau of Indian Standards
RAG	Retrieval-Augmented Generation
LLM	Large Language Model
NLP	Natural Language Processing
NER	Named Entity Recognition
JWT	JSON Web Token
PWA	Progressive Web App
DFD	Data Flow Diagram
UML	Unified Modeling Language
ERD	Entity Relationship Diagram
API Gateway	Public service boundary that routes/controls requests
Vector DB	Database/index used for similarity retrieval
Citation	Evidence pointer linking an answer to source material
Agent Router	Controlled logic deciding which supported AI tool to invoke

31. Final Architecture Decision Record

Decision

Use a two-service architecture: Node.js + Express as the public gateway/core backend and FastAPI + Python as the AI/GenAI microservice. Use MongoDB Atlas for application/chat data and Pinecone for vector retrieval. Use React/PWA and React Native as clients.

Rationale

This matches the supplied team architecture and separates business logic from AI logic, keeps the AI service behind the gateway, allows independent scaling and gives web/mobile clients one common API contract.

Build strategy

Build the smallest end-to-end vertical slice first: one verified BIS dataset subset → retrieval → grounded answer → citation → Node gateway → React UI. Then add recommendation, labs, certification, hallmarking, multilingual and mobile features. This reduces integration risk and aligns with the source plan's warning not to wait for 100% data before starting the AI pipeline.

Document status

This document is intended to serve as the team's master engineering/SRS blueprint. Where the uploaded plan did not specify an implementation detail, this document labels it as a proposed design rather than presenting it as an official BIS requirement.

END OF DOCUMENT